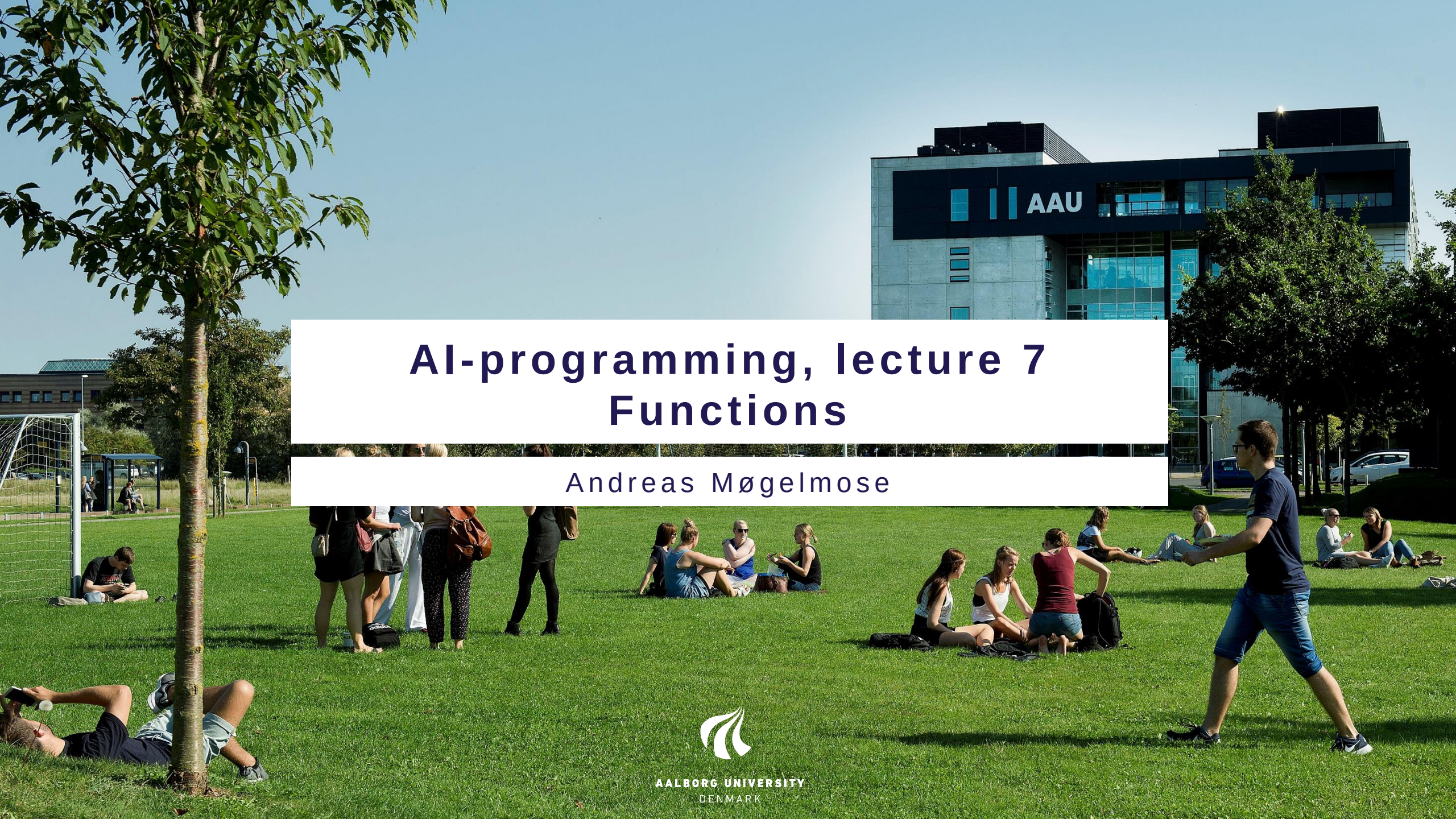


AI-programming, lecture 7 Functions

Andreas Møgelmose

AALBORG UNIVERSITY
DENMARK

Agenda

- Defining functions (input and output)
- Scope



Recap: Functions

```
my_string.lower()

move(7, 2)

QuitProgram()
```



Recap: Variables and lists

```
name = 'Bob'
age = 21
print(f"My name is {name}. I am {age} years old.")

print(20 % 7)

vegetables = ['carrot', 'lettuce', 'onion', 'radish']
vegetables.append('potato')
```



Recap: Loops

```
cars = ['audi', 'bmw', 'subaru', 'toyota']
cars = [car.title() for car in cars]

for car in cars[1:3]:
    print(car)
```



Recap: Branching

```
while True:
    screen.fill((100, 200, 100))
    s = datetime.now().second
    if s > 30 and enable_rect:
        pygame.draw.rect(screen, (200, 100, 100), (30, 70, 140, 60))
        if s == 59:
            enable_rect = False
    else:
        pygame.draw.ellipse(screen, (100, 100, 200), (30, 70, 140, 60))
```



Recap: Dicts/while/input

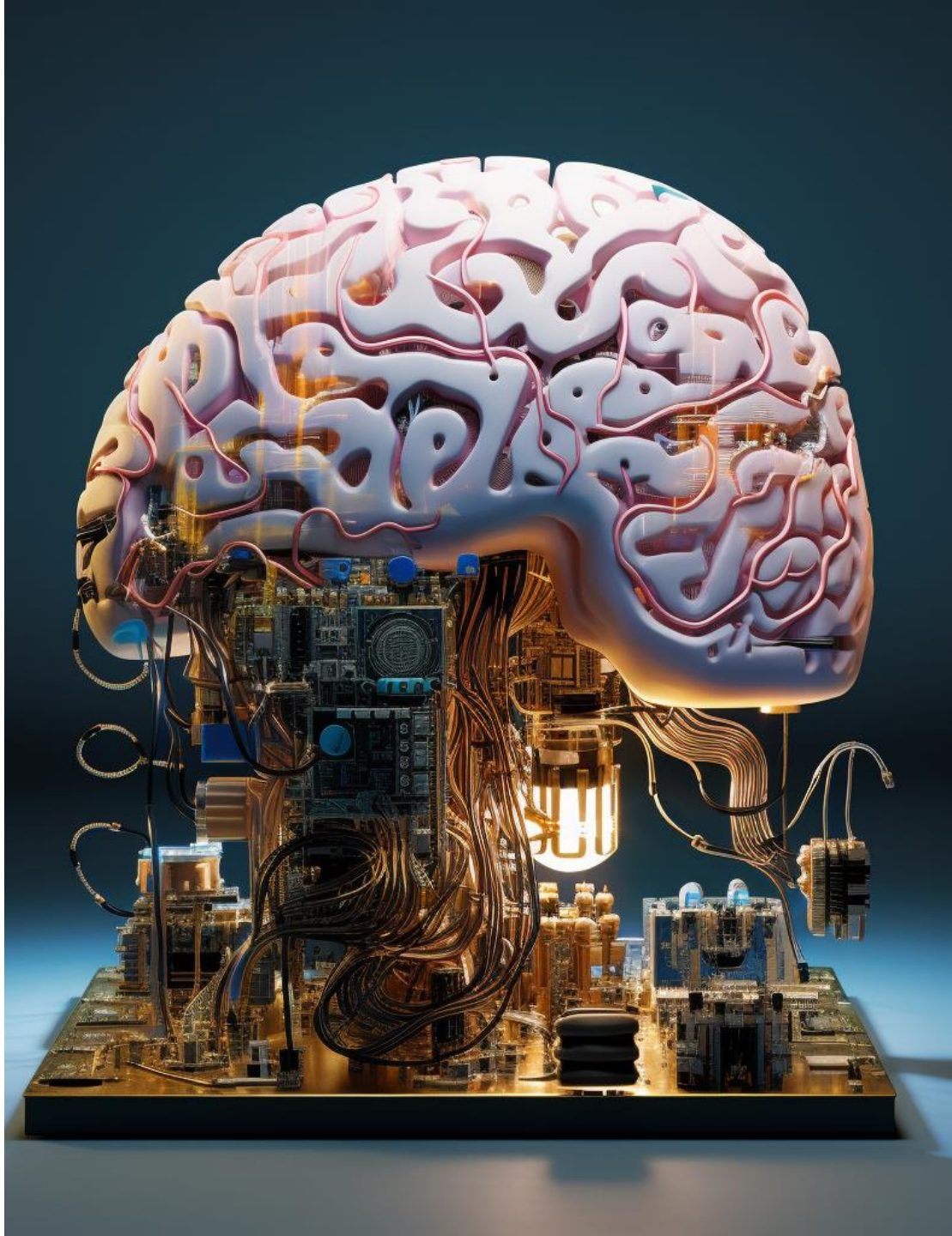
```
ages = {"Alice": 22, "Bob": 27, "Carol": 47}
name = input("Enter your name: ")
age = int(input("Enter your age: "))
ages[name] = age
print(ages)
```



Congratulations!

- You have now learned everything you need!
- With the tools we have covered in the last 5 lectures, you can implement any program in the world.

- …but it will neither be pretty nor easy to understand, so we're not quite done yet ☺



Control flow

- What might happen apart from linear execution?
  - An unconditional jump to some other line of code
  - Jump back to repeat a block of code
  - Jump ahead, skipping a block of code based on some condition
  - Jump ahead, selecting one of several possible blocks of code to execute based on a condition
  - Unconditionally jump to some other block of code, only to return at a later point in time
  - Stop the program

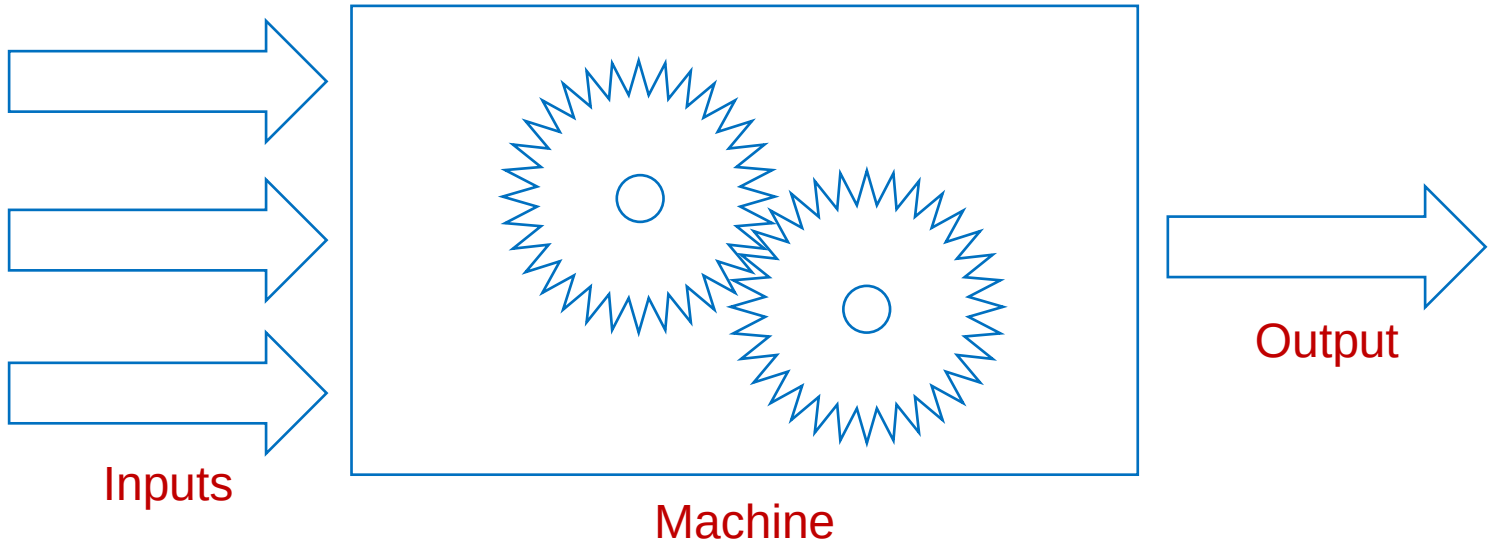
What lecture 3 was about: Looping!

What lecture 4 was about: Branching!

What this lecture is about: Functions!

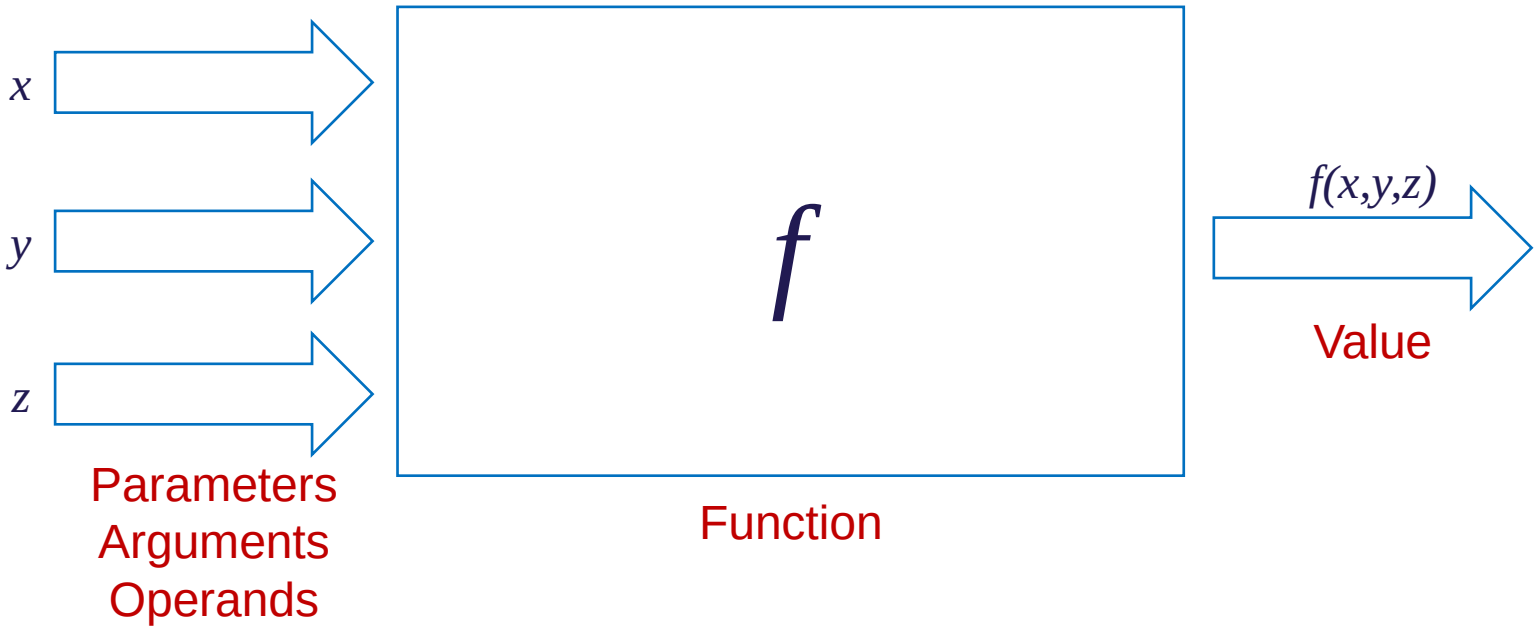
Defining functions

- A machine with input and output



Defining functions

- A mathematical function



You know functions

- Black boxes: You don’t see the content, but you call (use) them all the time!

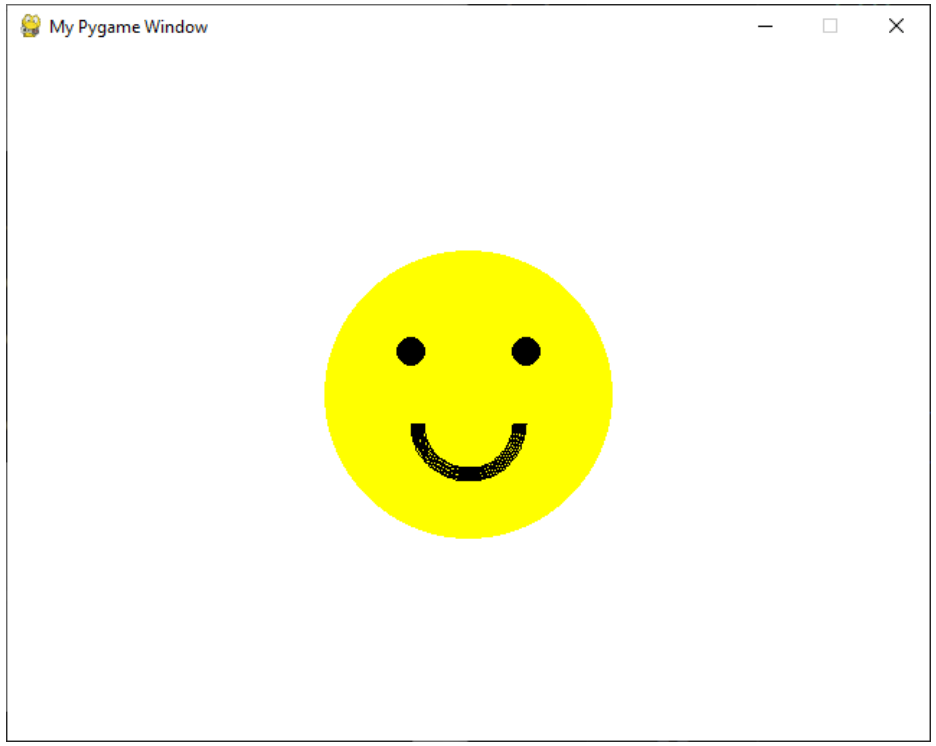
```
age = int(input("How old are you? "))
print(f"You are {age} years old")
```

```
for name, date in birthdays.items():
    print(f"{name} was born on {date}")
```

```
screen.fill((255, 255, 255))
box_position = (start_point[0] + datetime.now()
                start_point[1])
rect = box_position + box_size
pygame.draw.rect(screen, (0,0,0), rect)
pygame.display.flip()
```



Example




```
screen_width = 640
screen_height = 480
screen = pygame.display.set_mode((screen_width, screen_height))
pygame.display.set_caption("My Pygame Window")

def draw_smiley():
    position = (screen_width // 2, screen_height // 2)
    pygame.draw.circle(screen, (255, 255, 0), position, 100)
    pygame.draw.circle(screen, (0, 0, 0),
                       (position[0] - 40, position[1] - 30), 10)
    pygame.draw.circle(screen, (0, 0, 0),
                       (position[0] + 40, position[1] - 30), 10)
    pygame.draw.arc(screen, (0, 0, 0),
                    (position[0] - 40, position[1] - 20, 80, 80),
                    math.pi, 0, 10)

running = True
while running:
    # Handle events
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.fill((255, 255, 255))  # Fill the screen with white

    draw_smiley()

    pygame.display.flip()
```

Defining functions

Any name that would also be valid for a variable

```
def name(input1, input2, …):
    body

    return x
```

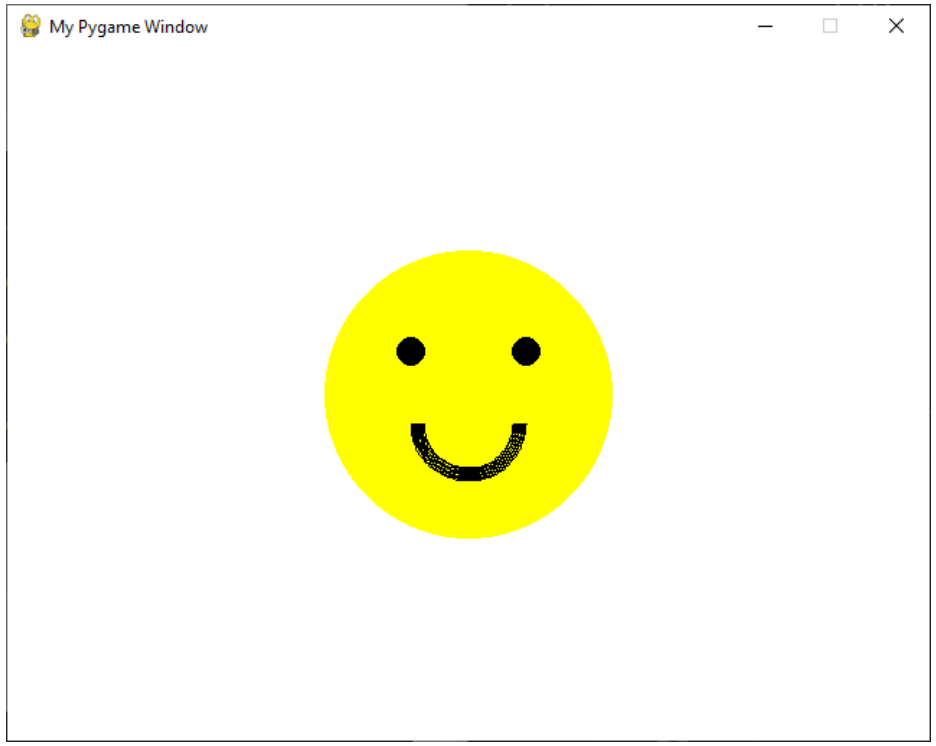
Any block of code will do

A series of variable declarations

Optional return



Example




```
screen_width = 640
screen_height = 480
screen = pygame.display.set_mode((screen_width, screen_height))
pygame.display.set_caption("My Pygame Window")

def draw_smiley():
    position = (screen_width // 2, screen_height // 2)
    pygame.draw.circle(screen, (255, 255, 0), position, 100)
    pygame.draw.circle(screen, (0, 0, 0),
                       (position[0] - 40, position[1] - 30), 10)
    pygame.draw.circle(screen, (0, 0, 0),
                       (position[0] + 40, position[1] - 30), 10)
    pygame.draw.arc(screen, (0, 0, 0),
                    (position[0] - 40, position[1] - 20, 80, 80),
                    math.pi, 0, 10)

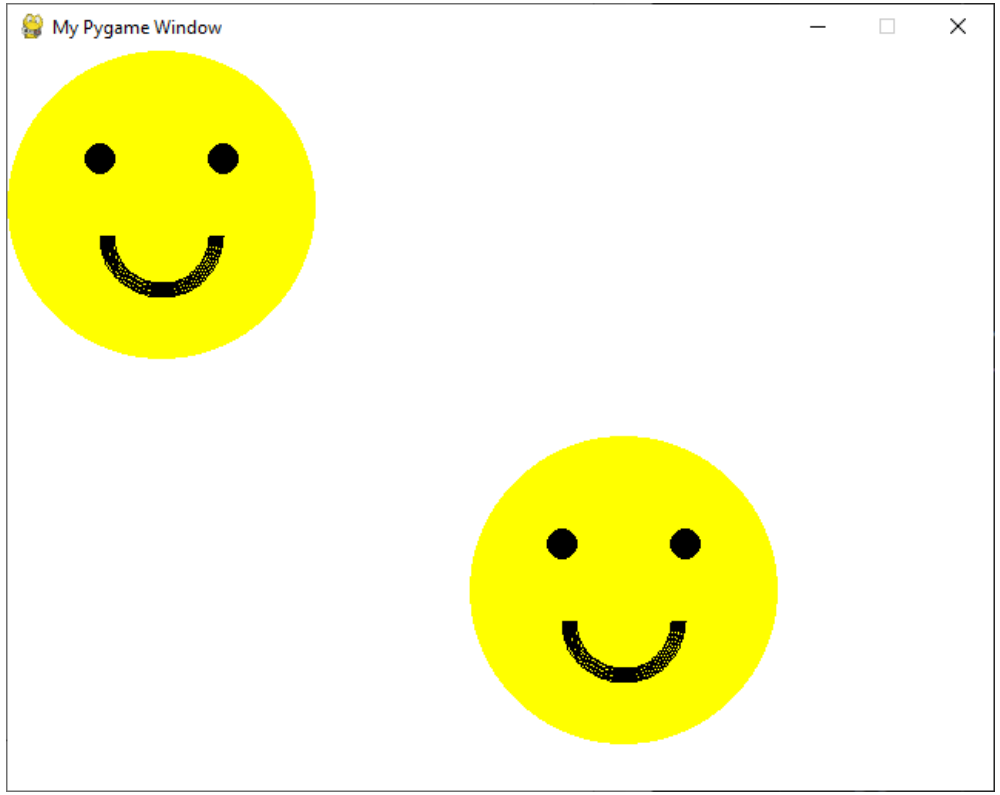
running = True
while running:
    # Handle events
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.fill((255, 255, 255))  # Fill the screen with white

    draw_smiley()

    pygame.display.flip()
```

Example with inputs




```
pygame.init()

screen_width = 640
screen_height = 480
screen = pygame.display.set_mode((screen_width, screen_height))
pygame.display.set_caption("My Pygame Window")

def draw_smiley(screen, position):
    pygame.draw.circle(screen, (255, 255, 0), position, 100)
    pygame.draw.circle(screen, (0, 0, 0),
                       (position[0] - 40, position[1] - 30), 10)
    pygame.draw.circle(screen, (0, 0, 0),
                       (position[0] + 40, position[1] - 30), 10)
    pygame.draw.arc(screen, (0, 0, 0),
                    (position[0] - 40, position[1] - 20, 80, 80),
                    math.pi, 0, 10)

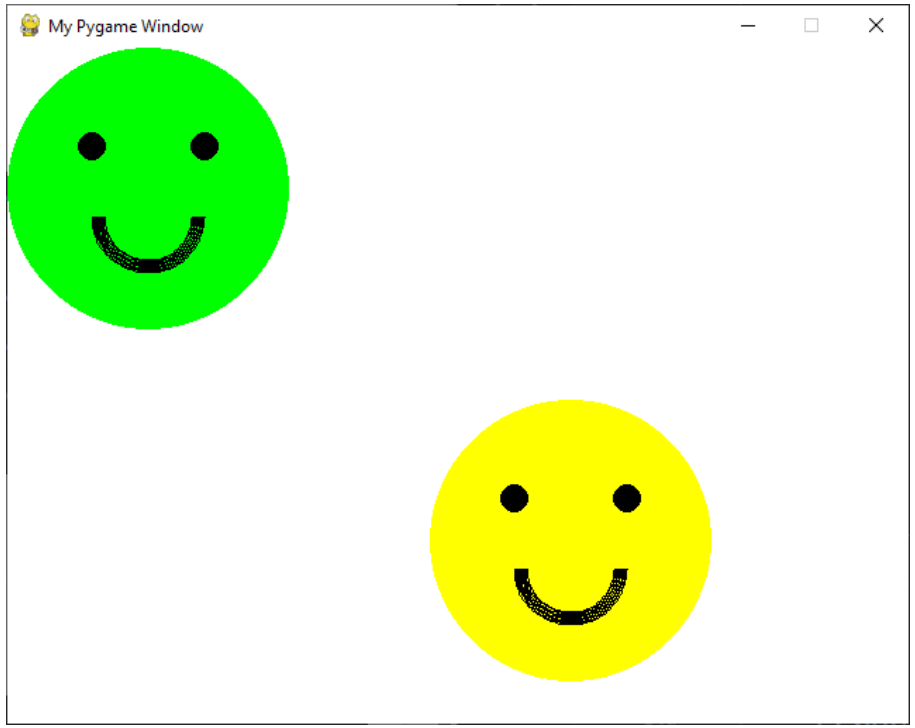
running = True
while running:
    # Handle events
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.fill((255, 255, 255))  # Fill the screen with white

    draw_smiley(screen, (100, 100))
    draw_smiley(screen, (400, 350))

    pygame.display.flip()
```

Example with inputs




```
pygame.init()

screen_width = 640
screen_height = 480
screen = pygame.display.set_mode((screen_width, screen_height))
pygame.display.set_caption("My Pygame Window")

def draw_smiley(screen, position = (100, 100), color = (255, 255, 0)):
    radius = 100
    pygame.draw.circle(screen, color, position, radius)
    pygame.draw.circle(screen, (0, 0, 0),
                       (position[0] - 40, position[1] - 30), 10)
    pygame.draw.circle(screen, (0, 0, 0),
                       (position[0] + 40, position[1] - 30), 10)
    pygame.draw.arc(screen, (0, 0, 0),
                    (position[0] - 40, position[1] - 20, 80, 80),
                    math.pi, 0, 10)
    return radius

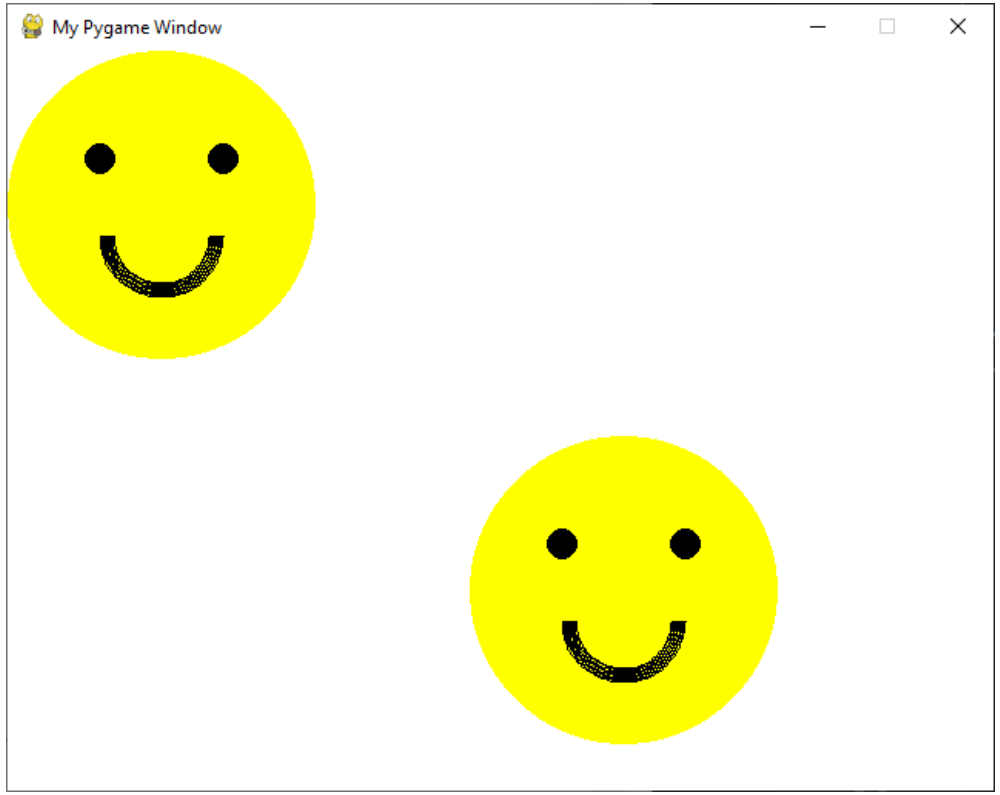
running = True
while running:
    # Handle events
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.fill((255, 255, 255))  # Fill the screen with white

    draw_smiley(screen, color = (0, 255, 0))
    draw_smiley(screen, (400, 350))

    pygame.display.flip()
```

Example with output




```
pygame.init()

screen_width = 640
screen_height = 480
screen = pygame.display.set_mode((screen_width, screen_height))
pygame.display.set_caption("My Pygame Window")

def draw_smiley(screen, position):
    radius = 100
    pygame.draw.circle(screen, (255, 255, 0), position, radius)
    pygame.draw.circle(screen, (0, 0, 0),
                       (position[0] - 40, position[1] - 30), 10)
    pygame.draw.circle(screen, (0, 0, 0),
                       (position[0] + 40, position[1] - 30), 10)
    pygame.draw.arc(screen, (0, 0, 0),
                    (position[0] - 40, position[1] - 20, 80, 80),
                    math.pi, 0, 10)
    return radius

running = True
while running:
    # Handle events
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.fill((255, 255, 255))  # Fill the screen with white

    radius = draw_smiley(screen, (100, 100))
    print(radius)
    print(draw_smiley(screen, (400, 350)))
```

Functions

- In Python, functions may return multiple things
  - They're automatically packed in a tuple

```
def my_favorite_numbers():
    return 7, 9, 13
print(my_favorite_numbers())
```

```
(7, 9, 13)
```

- Functions may have multiple `return` statements
  - They end execution of the function (exit the function) right when they are activated

```
def divide(a, b):
    if b == 0:
        print("Division by zero is not allowed")
        return None
    return a / b
```



Practical tips about functions

- If you have to scroll to read the entire function, it is too long! (some even say max 15 lines)
- Functions should do only one thing
- Choose descriptive names – usually a verb

- Slightly more advanced: Try to avoid side effects
  - Side effects are when functions manipulate external state, e.g. a global variable.



Scope and local variables

- Global variables
  - Defined outside any code block
- The (input) parameters of the function
- The local variables of the function
  - Defined inside the function

```
x = "a"
y = "b"
w = "c"


def my_function(x, y, z):
    w = "f"
    print(f"my_function: {x}, {y}, {z}, {w}")

my_function("e", x, w)
```

Lists vs. other arguments

```
def car_info(number_of_cars, cars):
    number_of_cars += 1
    cars.append("Toyota")


number_of_cars = 3
cars = ["Audi", "BMW", "Ford"]

car_info(number_of_cars, cars)

print(number_of_cars)
print(cars)
```

Lists vs. other arguments

```python
def car_info(number_of_cars, cars):
    number_of_cars += 1
    cars.append("Toyota")
    return number_of_cars


number_of_cars = 3
cars = ["Audi", "BMW", "Ford"]

number_of_cars = car_info(number_of_cars, cars)

print(number_of_cars)
print(cars)
```

Modules

module_import.py

```
1   import my_module
2
3   my_module.module_function()
```

my_module.py > ...

```
1   def module_function():
2       print("Yay, module_function!")
3
```

module_import.py

```
1   from my_module import module_function
2
3   module_function()
```



The main function

```
# Avoid globals like the plague!!

def main():
    print("Important stuff happening here!")

if __name__ == "__main__":
    main()
```



Exercise time!

AALBORG UNIVERSITY
DENMARK